Extending Computationally-Sound Symbolic Cryptography with PRGs in Lean

MAGGIE SIMMONS, The University of Waterloo, Canada

Symbolic cryptography allows reasoning about cryptographic protocols in an abstract way and is relatively easy to verify using proof assistants. However, it often lacks a connection to the computational aspects of real-world cryptography. Computationally-sound cryptography captures this connection much better, but it is often more complex and difficult to formally verify. Previous work formalized a translation between the two models in Lean, but did not include a model for key pseudorandom generators (PRGs), meaning the existing library's semantics were constructed with encryption as the sole primitive, which is impractical for real-world protocols [1]. By implementing an existing pen-and-paper model for PRGs [2], this extension enables the formal verification of practical garbling schemes with high-level symbolic proofs that automatically translate into rigorous computational indistinguishability guarantees.

CCS Concepts: • **Security and privacy** → **Logic and verification**; *Symmetric cryptography and hash functions.*

Additional Key Words and Phrases: Symbolic Cryptography, Formal Verification, Garbled Circuits, Computationally Sound Symbolic System

ACM Reference Format:
Maggie Simmons. 2026. Extending Computationally-Sound Symbolic Cryptography with PRGs in Lean. In *Proceedings of Software Verification (CS 747).* ACM, New York, NY, USA, 9 pages.

1 Introduction

The formal verification of cryptographic protocols bridges the gap between high-level security designs and mathematical certainty. While traditional "pen-and-paper" proofs are subject to human error (especially in complex protocols), interactive theorem provers like Lean 4 allow for machine-checked proofs of security. Historically, this verification has followed two primary paths: the **symbolic model** and the **computational model**.

Symbolic cryptography is significantly easier to formalize because it allows for reasoning that abstracts away the computational and probabilistic details of concrete implementations. In the symbolic model, operations like encryption are treated as formal terms (such as $Enc(k, m)$) rather than transformations of bitstrings. On the other hand, formalizing the computational model is notoriously difficult in proof assistants because it requires modeling probabilistic polynomial-time (PPT) interactive Turing machines and "non-negligible advantage". Both of these concepts lack a natural or practical representation in the formal verification setting, but models the real-world security of protocols more extensively.

By using a symbolic approach, cryptographers can leverage structural relations and co-induction to create proofs that are far more intuitive and amenable to machine checking. Fortunately, there exists a "best-of-both-worlds" approach - computationally-sound symbolic cryptography. After proving that the symbolic and computational view of a protocol are indistinguishable, cryptographers can formally verify their implementation in the symbolic setting while getting computationally-backed security guarantees.

This report details the implementation of an extension for a computationally-sound symbolic cryptography framework [1]. The goal of this extension is to expand a library designed for encryption and private-key primitives to include Pseudorandom Generators (PRGs). By doing so, we can

Author's Contact Information: Maggie Simmons, m8simmon@uwaterloo.ca, The University of Waterloo, Waterloo, Ontario, Canada.

CS 747, Waterloo, ON, Canada
2026.

formally reason about constructions like Garbled Circuits and other symmetric-key primitives that rely on the expansion of short, secret seeds into long, indistinguishable bitstrings.

2 Background

2.1 Game Hopping and Hybrid Arguments

In modern cryptography, security is rarely proven in a single step. Instead, we use **Game Hopping**. A “Game” is a formal experiment involving an adversary and a challenger.

- **Game 0 (Real):** The adversary interacts with the actual protocol.
- **Game n (Ideal):** The adversary interacts with a world where the security property is perfectly satisfied via some idealized oracle (e.g., all ciphertexts are replaced by random noise).

The proof consists of a sequence of intermediate "hops". If we can prove that a computationally bounded adversary cannot distinguish between Game i and Game $i + 1$, then by the triangle inequality of indistinguishability, the adversary cannot distinguish between the Real and Ideal worlds.

Hybrid Arguments are a specific type of game hopping used when a protocol uses many instances of a primitive (e.g., encrypting multiple messages). We “hop” by changing one instance at a time—replacing the first ciphertext with noise, then the second, and so on. The security of the whole system is thus reduced to the security of a single instance.

2.2 Challenges of Game Hopping in Lean

Implementing these arguments in Lean is significantly more difficult than on paper due to:

(1) **Monadic Complexity:** Cryptographic experiments are probabilistic. In Lean, these are often modeled in VCVio2 using an `OracleComp` monad. Managing the state, randomness, and oracle queries requires rigorous type-tracking that paper proofs usually abstract away.
(2) **Structural vs. Semantic Meaning:** One must prove that a *symbolic* change (replacing a sub-expression) corresponds to a *computational* change (a change in the probability distribution of bitstrings).
(3) **Variable Management:** In hybrid steps, one must carefully manage "fresh" variables and indices to ensure that the adversary’s view remains consistent across the hops without collisions.

2.3 Garbled Circuits

Garbled Circuits allow two parties to compute a function over their private inputs without revealing the inputs themselves. The logic gates of a boolean circuit are "garbled" by encrypting the truth table. PRGs (Pseudorandom Generators) make this practically feasible: they are used to expand short keys into the masks used to encrypt the gates. The security of the circuit relies on the fact that an adversary only ever learns the keys corresponding to the active paths, while all other "garbled" values remain indistinguishable from random strings.

2.4 IND-CPA and PRG Definitions

Following the framework established in the Baiyu Li paper, we define security via indistinguishability games:

- **IND-CPA (Indistinguishability under Chosen Plaintext Attack):** An encryption scheme is IND-CPA secure if an adversary, after choosing two messages m_0 and m_1, cannot distinguish between $Enc(k, m_0)$ and $Enc(k, m_1)$ with more than negligible advantage.

- **PRG (Pseudorandom Generator):** A PRG is a deterministic function $G : \{0,1\}^\lambda \to \{0,1\}^{L(\lambda)}$. It is secure if the distribution $\{s \leftarrow \{0,1\}^\lambda : G(s)\}$ is computationally indistinguishable from the uniform distribution $\{r \leftarrow \{0,1\}^{L(\lambda)} : r\}$. In our framework, we use a **distinguisher-based definition**: no polynomial-time algorithm D can tell the difference between a PRG output and a truly random string.

3 Implementation Accomplishments

Throughout the development of this extension, we have moved from basic definitions to complex soundness proofs. The following milestones were achieved:

(1) **Extended Expression Grammar:** Modified the core `Expression` inductive type to include `PRG` constructors, allowing the symbolic representation of seed expansion.
(2) **Symbolic Key Extraction (`extractKeys`):** Developed a recursive algorithm that determines which keys/seeds an adversary can "discover" by recursively opening encryptions they have keys for.
(3) **Monotonicity Proofs:** Proved that key extraction is non-decreasing and established a new greatest fixpoint for key inclusion. We formally showed that if an adversary performs a second pass of key extraction with the keys found in the first pass, they can only find a superset of the original keys. This ensures the `adversaryView` is stable.
(4) **Hiding Lemmas (`HidingOneKey.lean` & `HidingOnePrgSeed.lean`):**
    - Implemented the logic for a single "hop" in the hybrid argument.
    - Defined the reduction: a function that takes a PRG or Encryption oracle and "plugs" it into a larger symbolic expression.
    - Proved that if a seed is not in the `adversaryKeys`, replacing that seed's output with a fresh variable (representing random noise) is computationally sound.
(5) **Adversary View Normalization:** Integrated the `adversaryView` function, which hides parts of an expression that are computationally hidden, providing the bridge between symbolic equality and computational indistinguishability.

4 System Overview

Below is a brief description of the files in this framework, along with their dependencies. Most of these files are collections of related definitions and lemmas, some of them ending in major theorems, and others are logically grouped utility results. These files function as a library that can be used to prove that PRG-based Garbled Circuits are secure in future work.

4.1 VCVio2

This project depends on the VCVio library, to define computations that can query an oracle. The 'VCVio2' folder contains a fragment of this library, specifically the part that defines such computations as modified by [1].

4.2 PRGExtension/ComputationalIndistinguishability

This folder defines computational indistinguishability and develops some basic properties.

(1) `Def.lean` defines computational indistinguishability between two oracles. It also defines indistinguishability between two families of distributions. These definitions rely on an abstract notion of complexity which captures all allowed adversarial behavior(called PolynomialTime, as commonly assumed in cryptography). For more details, see the original paper.

(2) `Lemmas.lean` proves basic properties of indistinguishability, such as transitivity and symmetry. It also includes the lemma `IndistinguishabilityByReduction`, which shows how to use reductions to prove indistinguishability.

4.3 PRGExtension/Core

This folder contains general mathematical results used in the project.

(1) `Fixpoints.lean` proves a constructive version of the Knaster–Tarski theorem, showing the existence of fixpoints in a lattice of finite sets.
(2) `CardinalityLemmas.lean` contains auxiliary lemmas about the cardinality of sets.

4.4 PRGExtension/Expression

This module defines the expression language used in symbolic cryptography, based on [2]:

(1) `Defs.lean` defines expressions (type `Expression`).
(2) `Renamings.lean` defines the valid variable renaming.
(3) `SymbolicIndistinguishability.lean` defines symbolic indistinguishability between expressions (in particular it defines the ‘normalizeExpr‘ function that computes the normal form of an expression, and ‘adversaryView‘ which hides the parts of an expression that are not available to the adversary).

The ‘Expression/Lemmas‘ submodule contains various auxiliary lemmas:

(1) `NormalizeIdempotent.lean` proves that expression normalization is idempotent.
(2) `Renaming.lean` contains lemmas about commuting renaming and normalization.
(3) `HideEncrypted.lean` focuses on properties of `adversaryKeys`, such as bounds and dependence only on the keys used in an expression.

The ‘Expression/ComputationalSemantics‘ submodule contains the following files:

(1) `Def.lean` defines the computational semantics of an expression, i.e. a function that maps an expression (and an encryption scheme) to a distribution over bitstrings.
(2) `NormalizePreserves.lean` and `RenamePreserves.lean` prove that normalization and renaming do not change the computational semantics.
(3) `EncryptionIndCpa.lean` defines the notion of IND-CPA security for encryption schemes.
(4) `PrgSecurity.lean` defines the notion of distinguisher-based computationally indistinguishable PRGs.
(5) `HidingOneKey.lean` handles a computational hybrid step related to encryption security. It uses the IND-CPA property of the underlying encryption scheme to prove if the encryption scheme is secure, a "real" ciphertext is computationally indistinguishable from an "ideal" random one within an expression, provided the adversary does not possess the key.
(6) `HidingOnePrgSeed.lean` formalizes the "Idealization" of a PRG seed in another computational hybrid step. It proves that the output of a PRG can be replaced by truly random values without an adversary being able to tell the difference.
(7) `AdversaryView.lean` defines and reasons about the information an adversary can extract from a cryptographic expression without having the underlying keys. This is accomplished by moving from one hybrid game to another in a sequence of "hops".
(8) `Soundness.lean` proves the soundness theorem: if two expressions are symbolically indistinguishable, then their computational semantics (distributions over bitstrings) are computationally indistinguishable.

4.5 Note: SymbolicGarbledCircuitsInLean

Due to time constraints, I was unable to implement Garbled Circuits with PRGs. Since I did not modify the Encryption-Only Garbled Circuit formalization from the original framework associated with [1], I copied the implementation and the supporting encryption-only proofs from https://github.com/ravst/SymbolicCryptographyLean. Currently, these files are not included in the master SymbolicGarbledCircuitsInLean.lean file since they should be ignored for grading purposes. However, I included them in the full artifact to reflect my goal for creating a flexible library of computationally sound game-hopping proofs. For more information, please refer to the section on Future Work.

5 Formalization Methodology

In order to extend the existing `SymbolicGarbledCircuitsInLean` framework, I primarily relied on translating the pen-and-paper proofs established in [2] into Lean 4's dependent type theory. Because the transition from abstract cryptographic definitions to machine-checked logic is rarely one-to-one, this process required an iterative approach of interactive lemma discovery.

During the initial phases of the project, I took inspiration from the Haskell implementation provided alongside the original Baiyu Li paper. However, extracting reusable logic from the Haskell codebase was more difficult than pulling from a Rocq implementation, for example. The Haskell implementation was highly specialized for a concrete garbled circuit protocol rather than designed as a general-purpose symbolic framework. Additionally, the Haskell code relied heavily on standard pattern matching to simulate the symbolic execution of the circuit, but it completely omitted the computational security proofs. The sequence of hybrid arguments and game-hopping was left entirely to the pen-and-paper proof.

6 Implementation

The PRG extension represents the formalization of several interesting properties. Below are the parts of the proof I enjoyed the most (besides everything coming together in the final soundness proof). Most of the other properties, lemma, and theorems in this extension were extremely tedious due to what I call "rewrite" surgery in unfolding several layers of definitions (see later).

(1) **Constructive Knaster–Tarski and the Fixpoint Algorithm:** A primary achievement was the formalization of the Least Fixpoint Closure Algorithm for adversary knowledge, which serves as a constructive proof of the Knaster–Tarski theorem within our framework. The classical Knaster–Tarski theorem states that any monotone function on a complete lattice has a least fixpoint. In our formalization, the "lattice" is the power set of cryptographic keys present in the expression, represented rigorously as finite sets (`Finset`). Because the universe of keys in a given AST is finite, and we successfully proved that the key discovery function (`extractKeys`) is monotonically increasing (possessing more keys can only reveal an equal or greater number of keys), we were able to compute this fixpoint constructively. The algorithm iteratively applies the discovery function until the set stabilizes, providing a machine-checked, computable guarantee that the adversary's derivable knowledge is strictly bounded and perfectly well-defined.

(2) **Co-inductive Greatest Fixpoint Logic:** While the discovery of keys follows a least-fixpoint logic, the *indistinguishability* and, therefore, equivalence of expressions was formalized using a **Greatest Fixpoint** approach. This co-inductive definition allows the framework to reason about "cycles" of encryptions and ensures that two expressions are symbolically equivalent if no finite sequence of key-discoveries can reveal a structural difference. For more details on the relation of the Greatest and Least Fixpoints, see Appendix A

(3) **Implementation of Symbolic Substitution (`replacePRG`):** We implemented the `replacePRG` operation, which mirrors the "Idealization" step in garbled circuit proofs. This algorithm traverses the AST to find instances of a specific PRG evaluation (e.g., $G_0(s)$) and substitutes them with fresh variables. Without this, we would not be able to transition from a "Real" seed to a "Dummy" random value after a PRG was ruled computationally indistinguishable.

6.1 The Hybrid Argument Architecture

The computational soundness of the framework relies on a hybrid argument. Instead of proving that a complex symbolic expression is secure all at once, the framework defines a sequence of indistinguishable "hops." This architecture is distributed across three primary Lean modules:

- **`HidingOneKey.lean`:** This file forms the foundational unit of the hybrid argument. It formalizes the reduction from the IND-CPA security of the underlying encryption scheme to the symbolic framework. It proves that if an adversary does not possess a specific key k, replacing a single ciphertext encrypted under k with a "dummy" or hidden equivalent results in a computationally indistinguishable distribution. This module handles the granular probability math and the instantiation of the encryption oracle. If there was more time, a similar approach would have been completed `in HidingOneSeed.lean`.
- **`AdversaryView.lean`:** Building on the atomic hop, this module constructs the full hybrid sequence. It applies the `HidingOneKey` (and conceptually, the PRG idealization) lemmas iteratively over the entire set of keys and seeds that the fixpoint algorithm determined are inaccessible to the adversary. By peeling away the inaccessible components one by one, it bridges the gap between a single encrypted term and the normalized, fully hidden `adversaryView` of the entire protocol.
- **`Soundness.lean`:** This file ties the symbolic and computational models together. It leverages the transitive property of computational indistinguishability (`indTrans`) to chain the hybrid hops. The master proof follows a specific path: it first proves the real expression is computationally indistinguishable from its own idealized view ($Expr_1 \approx View(Expr_1)$). Because $Expr_1$ and $Expr_2$ are *symbolically indistinguishable*, their views are syntactically identical ($View(Expr_1) = View(Expr_2)$). Finally, it reverses the hybrid argument to bridge the view to the second expression ($View(Expr_2) \approx Expr_2$). This completes the triangle, proving $Expr_1 \approx Expr_2$.

7 Challenges and Limitations

Translating the abstract logic of [2] into Lean 4's kernel presented several challenges over pen-and-paper proofs.

7.1 Overformalization and Unification

The initial use of separate `HiddenG0` and `HiddenG1` constructors led to a combinatorial explosion of cases in inductive proofs. The framework struggled to reconcile these types during `adversaryView` normalization, leading to the use of a single "Hidden" constructor for efficient unification. Realizing and correcting this took most of the semester, but proofs in the rest of the framework were a lot more intuitive for me after the correction.

7.2 Incomplete PRG Idealization Lemmas

While the `HidingOneKey` (IND-CPA) lemmas were largely finalized, the PRG-specific lemmas in `HidingOnePrgSeed.lean` currently rely on `sorry` blocks. The primary difficulty lies in the structural induction required to prove that the `reductionToPrgOracle` is polynomial-time and

perfectly preserves the semantic indices of unrelated seeds without variable collisions. Lean's default automation struggled to resolve the unification of these fresh indices automatically. I also did not get around to proving/reasoning about collision resistance, so the hybrid argument really needed two more steps. Currently, the security of collision-resistant PRGs is axiomated in `PRGSecurity.lean`.

7.3 The ITE Unification Blocker

A recurring challenge was Lean's handling of *if-then-else* (`ite`) blocks within inductive proofs. Because the `extractKeys` function is defined via pattern matching on constructors (like `Enc` or `Pair`), wrapping a term in a conditional block makes it "opaque" to Lean's unification engine. This frequently caused the `apply` tactic to fail on logically sound steps, requiring the use of the `split` tactic or `apply_ite` lemmas to manually expose the underlying structure to the kernel.

7.4 Mutually Inductive Complexity

The `Expression` type in this framework is highly nested. Proving properties like monotonicity across `Pair`, `Enc`, and `PRG` constructors required complex mutual induction. Ensuring that the termination checker could verify these recursive functions—especially when dealing with the symbolic substitution of keys—required careful sizing of the expressions and lots of fine-grained rewriting and unfolding.

7.5 Managing Structural Substitution

In the symbolic model, "hiding" a ciphertext involves replacing it with a symbolic `Hidden` constructor. However, in the computational proof, this must be mapped to a probability distribution. The challenge lay in proving that the *syntactic substitution* of terms in Lean perfectly preserves the *semantic indices* of the underlying cryptographic variables, preventing "variable collisions" that would invalidate the computational reduction.

8 Conclusion and Future Work

This project is essentially a formally verified library for computationally-sound, symmetric-key cryptography. I was going to show its utility by using it to formalize a PRG-based garbled circuit implementation, but I ran out of time. Ultimately, I think what I accomplished in this project was a lot more valuable than "formalize Garbled Circuits but in a different language". The formal methods side of the cryptographic community currently has a lot of tools scattered around, but no unified framework. Lean's lack of utility for hybrid arguments was a major talking point at the conference I went to this semester (HACS), so if I continue working on this, I think it would be a very welcome addition to Lean's libraries.

As a result of all the recursive definitions, I was forced to become proficient at understanding Lean's interactive proving state and how to unfold, simplify, rewrite, and apply definitions and lemma precisely at the level I wanted. I learned a lot from this project and enjoyed the class as a whole!

8.1 Future Work

While this project successfully created the backbone for a more complex, practical, and computationally-sound cryptographic framework in Lean based on game-hopping proofs, there is a lot of work remaining.

8.1.1 Completing PRG Computational Security. As noted in the challenges, the computational security lemmas for PRG idealization (e.g., within `HidingOnePrgSeed.lean`) currently contain

`sorry` blocks. Completing these inductions and resolving the unification of fresh indices is the immediate next step to fully close the soundness proof. Additionally, future work must formally prove that the implemented PRGs satisfy collision-resistance properties, which are often implicitly assumed in pen-and-paper models.

8.1.2 Garbled Circuit Proof-of-Concept and Unified Framework. With the PRG extension in place, the framework is primed to formally verify a complete Garbled Circuit protocol. A direct follow-up project would involve implementing a concrete garbled circuit as a proof-of-concept, mirroring the Haskell implementation provided in [2]. Furthermore, this implementation should aim for a unified architecture that could choose between encryption-based and PRG-based garbled circuit constructions.

8.1.3 Refactoring Definitional Rigidity. The current structure of the AST and key definitions occasionally makes proving certain intuitively trivial properties about keys exceedingly difficult or impossible (particularly evident in the development of `HidingOneKey.lean`). Future iterations of this library should explore refactoring these core definitions, such as adopting a more flexible representation of "Hidden" terms, to ease the burden on Lean's unification engine and simplify structural inductions.

8.1.4 Evaluating the Scalability of the Oracle Architecture. The framework currently relies heavily on extending the `OracleComp` monad to introduce new cryptographic primitives. If the goal is to support an expanding library of protocols, this approach must be evaluated for long-term sustainability. Currently, adding new primitives (like PRGs alongside standard Encryptions) requires extensive modifications to the existing base oracles and probability distributions, which may not scale efficiently for more complex interactive protocols.

8.1.5 A Concrete Definition of PPT (Polynomial-Time). In the initial project proposal, investigating a concrete definition of Probabilistic Polynomial-Time (PPT) using S4 Kripke semantics was listed as a potential stretch goal. As the project evolved, it became clear that formally verifying a robust, concrete definition of PPT bound within Lean 4 is a massive undertaking—likely constituting an entire dissertation topic of its own. Currently, the framework relies on uninterpreted predicates for polynomial bounds; replacing these with a concrete calculus remains a major open problem for the formal cryptography community.

References

[1] Stefan Dziembowski, Grzegorz Fabiański, Daniele Micciancio, and Rafał Stefański. 2025. Computationally-sound symbolic cryptography in lean. Cryptology ePrint Archive, Paper 2025/1700. (2025). https://eprint.iacr.org/2025/1700.

[2] Baiyu Li and Daniele Micciancio. 2018. Symbolic security of garbled circuits. *IACR Cryptol. ePrint Arch.*, 141. http://eprint.iacr.org/2018/141.

A Extended Discussion of the Fixpoint Definitions for PRGs

We discovered that the Greatest Fixpoint definition needed to be updated when trying to correct the earlier overformalization issue.

This framework hinges on two mathematical fixpoints:

- **The Least Fixpoint (The PRG Closure):** This defines Adversary Knowledge. It is the constructive Knaster–Tarski algorithm that recursively expands what the adversary knows (if we have k, we can open $Enc(k, m)$ to find m. If we have a seed s, we can compute $G_0(s)$ and $G_1(s)$ to find new keys).
- **The Greatest Fixpoint (Symbolic Indistinguishability):** This defines Equivalence. It is a co-inductive relation that determines if two expressions "look the same" to the adversary.

We were forced to overhaul the Greatest Fixpoint definition for indistinguishability in the PRG extension for the following reasons.

A.1 Notions of "Hidden" PRGs

Originally, in standard encryption, the greatest fixpoint relation is straightforward: two ciphertexts look the same if the adversary does not have the key for either of them. They both just look like a generic `Hidden` term.

When PRGs were first introduced, I originally created specific constructors like `HiddenG0` and `HiddenG1` for PRG outputs whose seeds the adversary did not know.

However, this broke the Greatest Fixpoint logic. If the fixpoint relation requires strict structural matching, it would demand that a hidden G_0 output must exactly match another hidden G_0 output. However, the output of a secure PRG evaluated on an unknown seed is indistinguishable from a truly random string.

Therefore, the Greatest Fixpoint relation had to be rewritten again to allow cross-constructor equivalence. It had to be relaxed so that:

$$G_0(s) \equiv \texttt{VarK}(\text{fresh_dummy_idx}) \tag{1}$$

...if and only if $s \notin$ PRGClosure. The greatest fixpoint must equate an unreadable PRG branch with a completely fresh, independent random variable (`replacePRG`), which models the collision-resistant properties of secure PRGs.

A.2 The Dependency of Equivalence on the PRG Closure

In a pure encryption model, a key is just a key. But with PRGs, keys are generated dynamically. A seed s expands into two keys, $G_0(s)$ and $G_1(s)$, which might then be used to encrypt different gates in a Garbled Circuit.

The Greatest Fixpoint relation traverses the abstract syntax tree (AST) to check if two expressions are indistinguishable. To do this for PRGs, at every single node, the Greatest Fixpoint must query the Least Fixpoint (`PRGClosure`): "Does the adversary eventually discover the seed that generated this value?"

We had to change the Greatest Fixpoint definition to strictly take the final stabilized PRG Closure as a parameter. If we evaluated the indistinguishability relation using a partial closure (a state before the Least Fixpoint has finished settling), the framework might falsely conclude that a seed is hidden, evaluate it as a "Dummy" variable, and declare two expressions indistinguishable only for the adversary to legally discover the seed three steps later, breaking computational soundness.

A.3 Bridging the "Real" and "Ideal" Worlds

In the Haskell-based hybrid argument of [2], the simulator creates an "Ideal" circuit where inactive paths are literal random noise (`VarK fresh`). The "Real" circuit uses genuine PRG evaluations.

The core of the soundness proof requires proving:

$$\text{Real} \approx_{sym} \text{Ideal} \tag{2}$$

For the Greatest Fixpoint to successfully declare the Real and Ideal circuits symbolically indistinguishable, it cannot just be a simple recursive equality check. It had to be fundamentally redefined as a relation that says:

> "Two terms are equivalent if they are structurally identical, OR if one is a PRG evaluation with a seed outside the `PRGClosure` and the other is a fresh random variable not present anywhere else in the closure."

which is what we implemented in the PRG Extension.